Week3_예습과제_장서연

- 강화학습 개요 4-18
- 강화학습 기초 28-60
- 강화학습 기초2 그리드월드와 다이내믹 프로그래밍 67-75, 92-96

<강화학습 개념 정리 - 파이썬과 케라스로 배우는 강화학습>

강화학습 개요

> 📌 강화학습은 순차적으로 행동을 결정해야하는 문제

- 강화학습 = 행동 심리학과 강화학습, 머신러닝과 강화학습의 연관성
  - 행동심리학에서의 강화 = "동물이 이전에 배우지 않았지만 직접 시도하며 행동과 그 결과로 나타나는 좋은 보상 사이의 상관관계 학습"
    - 쥐가 점점 보상을 얻게 해주는 행동을 자주함.
    - 왜 보상이 주어지는지 이해 ✖️, 행동과 행동의 결과를 보상을 통해 연결
  - 머신러닝(아서 사무엘) = 기계가 일일이 코드로 명시하지 않은 동작을 데이터로부터 학습해 실행할 수 있도록 하는 알고리즘을 개발하는 연구 분야
    - 강화학습 = 정답 ❌, 주어진 데이터에 대해 학습 ❌
      주어진 <u>보상</u>을 통해 학습 ⭕
      - 보상 = 컴퓨터가 선택한 행동에 대한 환경의 반응, 간접적인 정답의 역할

✅ 스스로 학습하는 컴퓨터, 에이전트

- 에이전트 = 강화학습을 통해 스스로 학습하는 컴퓨터
- 사전지식 ❌ 상태에서 학습 → 강화학습의 장점

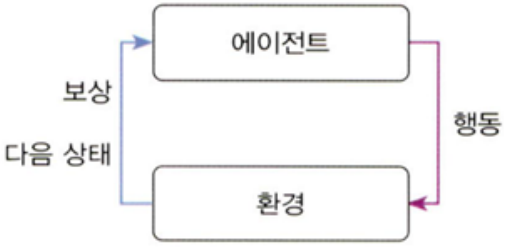


그림 1.4 에이전트와 환경의 상호작용

1. 자신이 놓인 환경에서 자신의 상태 인식
2. 행동
3. 환경 → 에이전트 보상, 다음 상태 알려줌
4. 반복 ⇒ 에이전트가 좋은 행동 학습

- 강화학습의 목적 : $\mathrm{Max} \sum$ (에이전트가 환경을 탐색하며 얻는 보상들)인 최적의 행동 양식/정책 학습
  - 보상 : 양수 / 처벌 : 음수
- 강화학습의 적용
  - 결정을 순차적으로 내려야 하는 문제
    - 다이나믹 프로그래밍, 진화 알고리즘 등 또한 이런 문제에 적용 가능
    - BUT 한계 有

✅ 순차적 행동 결정 문제

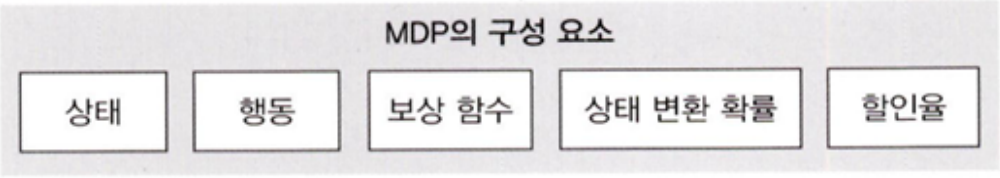
- 문제를 수학적으로 표현해야 에이전트가 학습하고 발전할 수 있음.
- **MDP :** 순차적 행동 결정 문제 정의 시 사용

1. 상태 (관찰)
   - 정적 요소, 동적 요소 등
   - e.g. 탁구 치기 학습 - 탁구공의 위치, 속도, 가속도
   - 방대한 상태 ⇒ 정보를 함수와 같은 형태로 근사하는 NN 사용
2. 행동 action
   - 처음에는 무작위로 행동, 학습하며 특정 행동들을 할 확률을 높임
   - e.g. 상, 하, 좌, 우

3. 보상 reward
    - 강화학습을 다른 머신러닝 기법과 다르게 만들어주는 가장 핵심 요소
    - 에이전트가 학습 가능한 유일한 정보
    - 에이전트에 속하지 않는 환경의 일부이므로 에이전트가 예측 불가능함.
4. 정책
    - 특정 상태가 아닌 모든 상태에 대해 어떤 행동을 할 지 정해야함
    - 최적 정책에 따라 행동했을 때 보상의 합을 최대치로 받을 수 있음.

강화학습 기초1 : MDP, 벨만 방정식

MDP - 문제 정의



📌 **구성요소**

1. 상태
2. 행동
3. 보상함수
4. 상태 변환 확률 (State Transition Probability)
5. 할인율 (Discount Factor)

1 상태

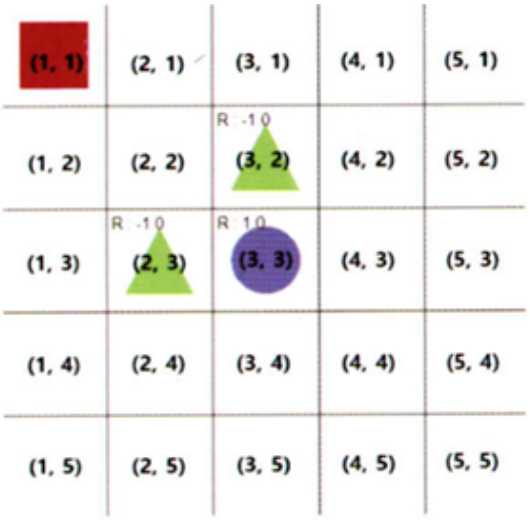


그림 2.2 그리드월드에서 상태는 좌표를 의미한다

- S : 에이전트가 관찰 가능한 상태의 집합 e.g. 센서 값
- 예제의 그리드 월드에서 상태 - 좌표값, 총 25개 존재
  $S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4), (x_5, y_5)\}$
- S_t : 어떤 t 에서의 상태, 정해져 있지 않음.
  - 상태는 시간에 따라 확률적으로 변함
  - 확률 변수

2 행동

- A : 에이전트가 상태 S_t에서 할 수 있는 가능한 행동의 집합 e.g. up,down,left,right
  $A = \{\text{up, down, left, right}\}$
- 한 문제 내에서 변하지 않음.

3️⃣ 상태 변환 확률

- 예상치 못한 요소를 포함해 에이전트가 특정 행동을 했을 때 어디로 이동할지 결정

$$\mathrm{P}_{ss'}^{a} = \mathrm{P}[\mathrm{S}_{t+1} = s' | \mathrm{S}_t = s, \mathrm{A}_t = a]$$

- 환경의 모델
- 환경은 에이전트가 행동을 취하면 상태변환 확률을 통해 다음에 에이전트가 갈 상태를 알려줌

4️⃣ 보상함수

- 에이전트가 학습할 수 있는 유일한 정보로서의 환경이 에이전트에게 주는 정보
- 보상함수에 대한 정의

$$r(s, a) = E[R_{t+1} | S_t = s, A_t = a]$$

  - 시간 t일 때 상태가 $S_t = s$ 이고 그 상태에서 행동 $A_t = a$를 했을 때 받을 보상에 대한 기댓값
  - $|$: 조건문에 대한 표현
  - R_{t+1} : 보상을 환경이 알려주는 것이기 때문에 <u>하나의 시간 단위</u>가 지난 다음에 보상 받음.
    - 하나의 시간단위 : <u>타임스텝(time step)</u>

5️⃣ 할인율

- 시간에 따라 할인하는 비율
  - 미래에 받은 보상은 현재의 가치로 환산한다면 그 크기가 적어짐.
  - 같은 보상이면 나중에 받을수록 가치가 줄어듦.
  - 할인율의 정의 : $\gamma \in [0, 1]$
  - 할인율을 고려한 미래 보상의 현재 가치 : $\gamma^{k-1} R_{t+k}$

6️⃣ 정책

- 모든 상태에서 에이전트가 할 행동
- 입력 : 상태, 행동 : 출력인 함수

$$\pi(a \mid s) = P[A_t = a \mid S_t = s]$$

수식 2.15 정책의 정의

-

가치함수 (value function)

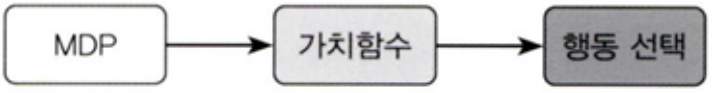


그림 2.10 에이전트는 가치함수를 통해 행동을 선택할 수 있다.

- 에이전트가 어떤 정책이 더 좋은 정책인지 판단하는 기준

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

- 보통 가치함수보단 에이전트가 선택할 각 행동의 가치를 직접적으로 나타내는 큐함수 사용

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s) q_\pi(s, a)$$

수식 2.26 가치함수와 큐함수 사이의 관계식

$$q_\pi(s, a) = E_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

벨만 방정식

- 현재 상태의 가치함수와 다음 상태 가치함수 사이의 관계식

1. 기대 방정식 - 특정 정책을 따라갔을 때

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

$$v_\pi(s) = \sum_{a \in A} \pi(a \mid s) \left(r_{(s,\, a)} + \gamma \sum_{s' \in S} P_{ss'}^{a} v_\pi(s') \right)$$

수식 2.32 계산 가능한 벨만 방정식

2. 최적 방정식 - 최적의 가치함수를 받게하는 정책과 가식

$$v_*(s) = \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

강화학습 기초2 : 그리드월드와 다이내믹 프로그래밍

- 순차적 행동 결정 문제 진행 과정

1. 순차적 행동 문제를 MDP로 전환한다.
2. 가치함수를 벨만 방정식으로 반복적으로 계산한다.
3. 최적 가치함수와 최적 정책을 찾는다.

그림 3.1 순차적 행동 결정 문제를 풀어나가는 과정

- 다이내믹 프로그래밍
  - 작은 문제가 큰 문제 안에 중첩돼있을 때 작은 문제의 답을 다른 작은 문제에서 이용
  - 여러번으로 쪼개 가치함수를 구함.

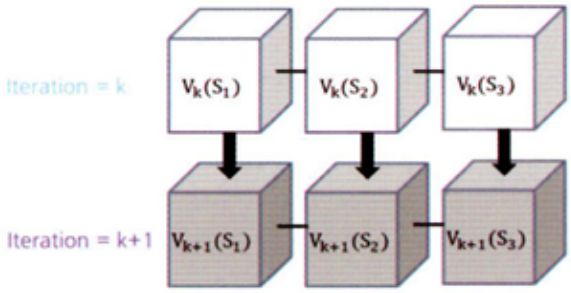


그림 3.2 다이내믹 프로그래밍의 개념

  - 벨만 기대방정식 : 정책 이터레이션
  - 벨만 최적 방정식 : 가치 이터레이션
- 정책 이터레이션
  - 다이내믹 프로그래밍으로 벨만 기대 방정식 풀기
  - 정책 평가 + 정책 발전
    - 정책 평가 : 현재 정책에 대한 참 가치함수 구함
    - 정책 발전 : 정책 평가 시 벨만 기대 방정식을 통해 구한 가치함수를 토대로 최대의 보상을 얻게 하는 행동 선택, 탐욕 정책 발전

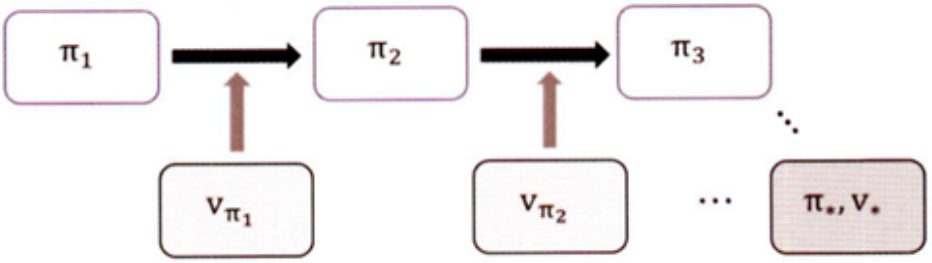


그림 3.17 정책 이터레이션에서 정책과 가치함수가 발전해나가는 과정

$$v_\pi(s) = E_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

수식 3.12 벨만 기대 방정식을 통해 구하는 것은 현재 정책에 대한 참 가치함수다

- 가치 이터레이션
  - 벨만 최적 방정식으로 순차적 행동 결정 문제에 접근
  - 최적 정책 가정
  - 정책이 직접적으로 주어지지 않으며 행동의 선택은 가치함수를 통해 이뤄짐

$$v_*(s) = \max_a E[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

수식 3.13 벨만 최적 방정식을 통해 구하는 것은 최적 가치함수다

- 다이내믹 프로그래밍의 한계
  - 계산 복잡도, 차원의 저주, 환경에 대한 완벽한 정보 필요
    ⇒ 모델 없이 학습하는 강화학습

Playing Atari with Deep Reinforcement Learning

1. Introduction

논문이 다루는 분야

- DQN (Deep Q-Network)

- Deep Reinforcement Learning
- Computer Vision + Reinforcement Learning 결합

해당 task에서 기존 연구 한계점

- 기존 Q-learning 문제: Q-table 필요 $Q(s, a)$
- Atari 환경: state가 이미지

  예: 84×84×4 → 가능한 state 개수: 256^{28224} → 사실상 무한, tabular 방법 불가능
- neural network로 Q-learning 학습 시 발생하는 문제:
  1. 데이터 간 strong correlation
  2. target 값이 계속 변함
  3. 학습 불안정 (divergence 발생)

논문의 contributions

1. CNN을 이용하여 Q-function 근사
   a. 픽셀 입력 → 행동 가치 출력
2. Experience Replay 도입
   - 데이터 상관관계 문제 해결, 학습 안정성 증가
3. Target Network 도입
   - moving target 문제 해결, 학습 안정화
4. End-to-end RL 가능성 제시
   - vision + control 통합 학습 가능함을 증명
5. 하나의 구조로 다양한 게임 해결 가능함 입증
   -

2. Related Work

- 기존 Reinforcement Learning : Q-learning 업데이트 식
  $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma)$
  $\max Q(s', a') - Q(s, a))$
  - state space가 작을 때만 효과적으로 동작한다.

- 이전 연구에서도 neural network를 이용하여 Q-function을 근사하려는 시도가 있었지만 학습이 매우 불안정하고 발산 문제가 발생
- CNN은 이미지 처리 분야에서 매우 좋은 성능을 보였지만 RL과 직접 결합된 연구는 많지 않았다.
- DQN의 차별점은 perception(이미지 특징 추출)과 decision making(행동 선택)을 하나의 모델로 통합하여 동시에 학습한다는 점이다.

3. 제안 방법론

Main Idea

- Q-learning의 Q-function을 CNN으로 근사
- 입력: 게임 화면(state)
- 출력 : 각 action에 대한 Q-value
- 학습 시 target 값 : $y = r + \gamma, maxQ(s', a'; \theta^-)$ 로 계산
- loss : $L(\theta) = (y - Q(s, a; \theta))^2$ 형태의 MSE loss

Contribution

- Experience Replay는 transition (s, a, r, s') 데이터를 replay buffer에 저장
- random sampling하여 학습 데이터 간 상관관계를 줄이고 sample efficiency를 증가시킴.
- Target Network는 Q-network와 동일한 구조의 별도 네트워크 θ^-를 두고 일정 주기마다 파라미터를 복사하여 target 값을 안정적으로 유지
- CNN architecture는 입력으로 84×84 grayscale frame 4장을 stack하여 속도 정보를 포함하도록 구성
  - Conv(32 filters, 8×8 kernel, stride 4) → Conv(64 filters, 4×4 kernel) → Conv(64 filters, 3×3 kernel) → Fully Connected layer(512 units) 구조
  - 최종 출력은 가능한 action 개수만큼의 Q값

4. 실험 및 결과

💡 실험의 목적은 raw pixel만으로 학습이 가능한지, 다양한 게임에서 동일한 구조가 general하게 작동하는지, human 수준 성능에 도달하는지를 검증하는 것이다.

Dataset

- Atari 2600 환경인 Arcade Learning Environment
- 실험에 사용된 게임 : Breakout, Pong, Space Invaders, Seaquest, Beam Rider 등

Baseline

- 기존 reinforcement learning 방법들과 human performance를 기준으로 성능 비교

결과

- 많은 Atari 게임에서 human 수준 이상의 성능을 달성
- 게임마다 다른 feature engineering 없이 동일한 모델 구조를 사용
- 이는 하나의 architecture가 다양한 task에 적용될 수 있음을 보여주며 general AI 방향성을 제시

5. 결론 (배운점)

연구의 의의

- perception과 control을 하나의 모델에서 end-to-end로 학습할 수 있음
- 기존에는 vision과 RL이 분리되어 설계되었지만 DQN은 representation learning을 통해 state representation과 policy 학습을 동시에 수행
- deep learning이 reinforcement learning 문제 해결에 효과적일 수 있음 제시, 이후 연구 발전의 기반이 되었다.

한계점

- sample inefficiency 문제로 인해 많은 환경 interaction이 필요하며 학습이 여전히 불안정할 수 있고 hyperparameter에 민감하다. 또한 DQN은 discrete action space에 적합하며 continuous action 문제에는 적용이 어렵다.

개인적으로 배운점

- 좋은 모델은 반드시 복잡할 필요가 없으며 데이터 분포와 representation 설계가 매우 중요하다는 점을 배웠다.